Recursion and the Stack: Key Terms

Name: _____

Date: _____

Write a definition in your own words. Add an example or a quick sketch if it helps.

Recursion

Base case

Recursive case

Stack

Push

Pop

LIFO (Last In, First Out)

Call stack

Stack frame

Stack overflow

Trace 1: power(2, 3)

Name: _____

Date: _____

Follow along with the 'Tracing power(2, 3)' slides.

power.py

```
1 def power(a, b):
2     if b == 1:
3         return a
4     else:
5         return a * power(a, b-1)
6
7 print(power(2, 3))
```

1. Predict

Before tracing, what do you think power(2, 3) prints?

2. Memory diagram

Draw one box per call. Inside, write each variable and its value (e.g. a → 2).

3. Call stack

Push a frame for each call. Cross it off when it returns.

top of the stack (running now) ↑

bottom of the stack (first call)

Base case reached when: _____ **Actual output:** _____

Return values, from the top of the stack down: _____

Trace 2: function(2)

Name: _____

Date: _____

Trace it yourself first, then check against the 'Tracing function(2)' slide.

function.py

```
1 def function(n):
2     if n == 4:
3         return n
4     else:
5         return 2 * function(n+1)
6
7 print(function(2))
```

1. Predict

Before tracing, what do you think function(2) prints?

2. Memory diagram

Draw one box per call. Inside, write each variable and its value (e.g. a → 2).

3. Call stack

Push a frame for each call. Cross it off when it returns.

top of the stack (running now) ↑

bottom of the stack (first call)

Base case reached when: _____ **Actual output:** _____

Return values, from the top of the stack down: _____

Trace 3: function(4, 3)

Name: _____

Date: _____

Watch how x and y change in each frame.

function.py

```
1 def function(x, y):
2     if x == 0:
3         return y
4     else:
5         return function(x-1, x+y)
6
7 print(function(4, 3))
```

1. Predict

Before tracing, what do you think function(4, 3) prints?

2. Memory diagram

Draw one box per call. Write x and y for each call. What happens to x + y?

3. Call stack

Push a frame for each call. Cross it off when it returns.

top of the stack (running now) ↑

bottom of the stack (first call)

Base case reached when: _____ **Actual output:** _____

Return values, from the top of the stack down: _____

Practice: fac(4)

Use the same steps on a new problem.

Name: _____

Date: _____

factorial.py

```
1 def fac(a):
2     if a == 1:
3         return 1
4     else:
5         return a * fac(a-1)
6
7 print(fac(4))
```

1. Predict

Before tracing, what do you think fac(4) prints?

2. Memory diagram

Draw one box per call. Inside, write each variable and its value (e.g. a → 2).

3. Call stack

Push a frame for each call. Cross it off when it returns.

top of the stack (running now) ↑

bottom of the stack (first call)

Base case reached when: _____ **Actual output:** _____

Return values, from the top of the stack down: _____